

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

**по выполнению курсового проекта
по дисциплине «Программная инженерия» для направления
подготовки 09.03.04 «Программная инженерия» направленность
(профиль) «Разработка и сопровождение программного обеспечения»**

Ставрополь 2026

ЧАСТЬ 1. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКАЯ

1.1. Цель и задачи курсового проекта

Цель курсового проекта: Закрепление и углубление теоретических знаний, полученных в курсе «Программная инженерия», а также формирование практических навыков проектирования, разработки и документирования программного обеспечения на всех этапах жизненного цикла от бизнес-анализа до развертывания готового продукта с использованием архитектурного паттерна PCMEF (Presentation-Control-Mediator-Entity-Foundation).

Задачи курсового проекта:

1. Провести предпроектное обследование и бизнес-анализ предметной области.
2. Разработать модель требований к программному обеспечению.
3. Спроектировать архитектуру программной системы на основе паттерна PCMEF.
4. Реализовать ядро системы с соблюдением принципов SOLID и слоевой архитектуры
5. Обеспечить качество кода через тестирование и рефакторинг.
6. Разработать пользовательский интерфейс в соответствии с выбранной траекторией.
7. Обеспечить безопасность и управление транзакциями.
8. Предоставить API для интеграции с внешними системами (для соответствующих траекторий)
9. Выполнить развертывание системы
10. Подготовить полный комплект проектной документации

1.2. Организация выполнения курсового проекта

1.2.1. Сроки и этапы (18 недель)

Неделя	Этап	Вес в оценке
1-2	Этап 0: Инициация	5%
3-4	Этап 1: Требования	10%
5-6	Этап 2: Архитектура	10%
7-8	Этап 3: Проектирование БД	10%
9-10	Этап 4: Детальное проектирование	10%
11-12	Этап 5: Реализация ядра	15%
13-14	Этап 6: Рефакторинг	10%
15-16	Этап 7: Интерфейс	15%
17-18	Этап 8: Завершение	15%

1.2.2. Форма организации работы

Индивидуальное выполнение:

- Каждый студент выполняет собственный проект
- Тема выбирается из утвержденного списка или предлагается

самостоятельно

- Полная персональная ответственность за все этапы

1.2.3. Требования к репозиторию

Обязательное использование Git для управления версиями:

<https://github.com/student-name/course-project>

Структура репозитория:

```

docs/
├── 01-business-model/
│   ├── README.md           # Обзор этапа
│   ├── context-diagram.md  # IDEF0
│   ├── buc-diagram.md      # BUC-диаграмма
│   ├── glossary.md         # Глоссарий
│   └── images/             # Изображения
├── 02-requirements/
│   ├── README.md
│   ├── use-case-diagram.md
│   ├── domain-model.md
│   ├── use-case-specifications.md
│   └── images/
├── 03-architecture/
│   ├── README.md
│   ├── pcmeff-diagram.md
│   ├── interfaces.md
│   └── images/
├── 04-database/
│   ├── README.md
│   ├── er-diagram.md
│   ├── ddl.sql
│   └── images/
├── 05-design/
│   ├── README.md
│   ├── sequence-diagrams.md
│   └── images/
├── 06-implementation/
│   ├── README.md
│   ├── code-structure.md
│   ├── tests.md
│   └── images/
├── 07-ui/
│   ├── README.md
│   ├── screenshots.md
│   └── images/
├── 08-final/
│   ├── technical-specification.md
│   ├── user-guide.md
│   ├── admin-guide.md
│   ├── presentation.pptx
│   └── images/
├── images/                 # Общие изображения
└── git-stats.png

```

Требования к документированию в Markdown:

1. Все текстовые документы должны быть оформлены в формате Markdown (.md)

2. В корне репозитория обязательно наличие файла README.md с описанием проекта

3. Документация по каждому этапу должна храниться в отдельной папке docs/XX-этап/

4. Все изображения (диаграммы, скриншоты) должны храниться в папке docs/images/

5. Ссылки на изображения в Markdown должны быть относительными: ![описание](images/file.png)

Требования к визуализации Git-статистики:

1. В файл README.md обязательно включить раздел "Статистика разработки"

2. Раздел должен содержать скриншоты статистики из GitHub Insights (или аналогичного инструмента):

- График активности коммитов (Commit Activity)
- Тепловая карта (Punch Card) - распределение по дням/часам

3. Скриншоты должны быть сделаны на момент сдачи проекта (финальная статистика)

4. Изображения сохраняются в папку docs/images/ с названием git-stats-*.png

5. В разделе также привести текстовые метрики:

- Общее количество коммитов
- Период разработки (дата первого и последнего коммита)

Пример оформления раздела статистики:

```
```markdown
```

```
Статистика разработки
```

```
Метрики Git
```

```
- Всего коммитов: 47
```

- Период: 01.03.2026 - 30.05.2026
- Средняя частота: 2.9 коммита/неделю

### График активности

![Активность коммитов](images/git-commit-activity.png)

### Тепловая карта

![Распределение по времени](images/git-punch-card.png)

### 1.3. ВЫБОР ТРАЕКТОРИИ ПРОЕКТА

#### 1.3.1. Общая концепция: PCMEF как единая архитектурная основа

В основе всех траекторий курсового проектирования лежит архитектурный паттерн PCMEF (Presentation-Control-Mediator-Entity-Foundation). Этот паттерн является концептуальным каркасом, который:

- Обеспечивает четкое разделение ответственности между компонентами;
- Гарантирует направленность зависимостей сверху вниз;
- Позволяет тестировать каждый слой изолированно;
- Адаптируется под различные среды выполнения.

Структура PCMEF:

Слой	Английское название	Русское название	Ответственность
P	Presentation	Представление	Интерфейс пользователя, отображение данных
C	Control	Управление	Обработка запросов, координация, валидация
M	Mediator	Посредник	Бизнес-логика, транзакции, правила

E	Entity	Сущность	Бизнес-объекты, состояние, методы предметной области
F	Foundation	Фундамент	Доступ к данным, инфраструктура, репозитории

Правило зависимостей: Зависимости направлены строго вниз. Вышележащие слои могут зависеть от нижележащих, но не наоборот. Это обеспечивает стабильность архитектуры.

### 1.3.2. Доступные траектории выполнения

Траектория	Фокус	Технологии (например)	Оценка
А. Desktopная	Толстый клиент, локальная БД	Java Swing/JavaFX, SQLite/PostgreSQL	3-4
Б. Веб-ориентированная	Тонкий клиент, серверное приложение	JSP/Servlets/Spring MVC, PostgreSQL	5
В. Мобильная	Нативное/кроссплатформенное мобильное приложение	Android (Java/Kotlin), Flutter, React Native + Java Backend	5
Г. Enterprise	Полный стек + API + контейнеризация	Full Stack, REST, Docker, Cloud	При успешной защите, экзамен автомат

### 1.3.3. Траектория А: Desktop-ориентированная

Фокус: Разработка настольного приложения с графическим интерфейсом и локальной или удаленной базой данных.

Реализация PCMEF:

- Все слои (P, C, M, E, F) находятся в одном JVM-процессе
- Взаимодействие между слоями — прямые вызовы методов
- Presentation-слой реализован на Swing или JavaFX

Технологический стек:

Компонент	Технология
Язык	Java 17+
UI	Java Swing / JavaFX
БД	SQLite / PostgreSQL
Доступ к данным	JDBC / JPA
Сборка	Maven / Gradle

Обязательные требования:

- Desktop UI с 5+ экранами
- Аутентификация пользователей
- CRUD-операции для основных сущностей
- Модульное тестирование (JUnit, покрытие >40%)
- Полная документация (Markdown)

#### 1.3.4. Траектория Б: Веб-ориентированная

Фокус: Разработка веб-приложения с тонким клиентом и серверной частью на Java.

Реализация PCMEF:

- Presentation-слой в браузере (HTML/CSS/JavaScript)
- Control, Mediator, Entity, Foundation на сервере
- Взаимодействие через HTTP/JSON (REST API)

Технологический стек:

Компонент	Технология
-----------	------------



Серверный язык	Java 17+
Серверный фреймворк	Spring Boot / Servlets + JSP
Клиент	HTML, CSS, JavaScript
БД	PostgreSQL
ORM	Spring Data JPA / Hibernate
Сборка	Maven / Gradle

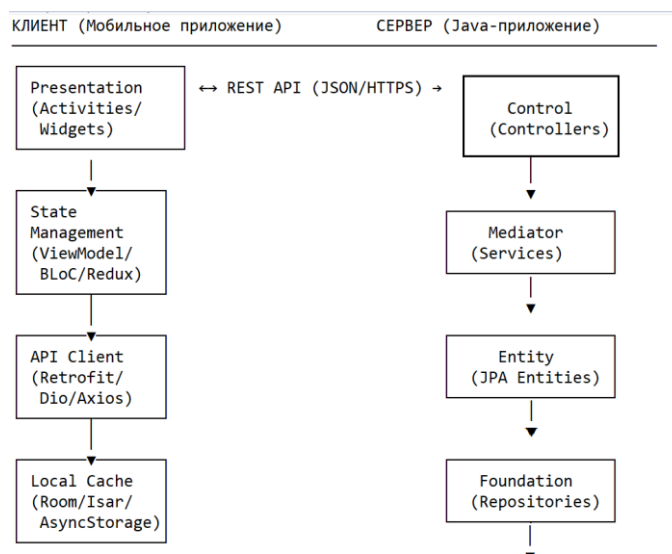
Обязательные требования:

- Web UI с 5+ страницами
- Аутентификация и авторизация (роли)
- REST API для взаимодействия
- Модульное тестирование (JUnit, покрытие >40%)
- Развертывание на сервлет-контейнере (Tomcat)

### 1.3.5. Траектория В: Мобильная разработка

Фокус: Разработка клиент-серверного мобильного приложения с бэкендом на Java.

Реализация PCMEF (адаптированная):



Доступные технологические стеки для клиента:

Вариант	Технологии	Особенности
Android Native	Kotlin + Jetpack Compose/XML + Retrofit + Room	Нативный Android, доступ ко всем API
Flutter	Dart + Widgets + Dio + SQLite	Кроссплатформенный, единая кодовая база
React Native	TypeScript + React Native + Axios + AsyncStorage	Кроссплатформенный, экосистема JS

Серверная часть (единая для всех вариантов):

Компонент	Технология
Язык	Java 17+
Фреймворк	Spring Boot
БД	PostgreSQL
ORM	Spring Data JPA
API	REST + OpenAPI/Swagger
Безопасность	JWT + Spring Security
Сборка	Maven / Gradle

Обязательные требования:

- Мобильное приложение с 5+ экранами
- Серверная часть на Java (Spring Boot)
- REST API (8+ эндпоинтов) с документацией OpenAPI
- Аутентификация через JWT
- Локальное кэширование (оффлайн-режим)
- Модульное тестирование сервера (покрытие >40%)

### 1.3.6. Траектория Г: Enterprise (полный стек)

Фокус: Разработка полноценной корпоративной системы с несколькими типами клиентов и контейнеризацией.

Реализация PCMEF:

- Desktop и Web клиенты (оба типа интерфейсов)
- REST API для интеграции
- Полный набор слоев на сервере с расширенной безопасностью
- Контейнеризация всех компонентов

Технологический стек:

Компонент	Технология
Desktop UI	JavaFX / Swing
Web UI	React / Angular / Vue + TypeScript
Сервер	Java + Spring Boot
API	REST + GraphQL (опционально)
БД	PostgreSQL
Кэш	Redis (опционально)
Контейнеризация	Docker + docker-compose
CI/CD	GitHub Actions / GitLab CI
Безопасность	Spring Security + JWT + OAuth2

Обязательные требования:

- Два типа интерфейсов (Desktop + Web)
- Полноценное REST API (15+ эндпоинтов)
- Документация OpenAPI (Swagger)
- Расширенная безопасность (роли, JWT, HTTPS)
- Контейнеризация (Docker + docker-compose)
- Интеграционные тесты
- CI/CD pipeline

#### 1.3.7. Таблица сравнения и рекомендации по выбору

Критерий	А: Десктоп	Б: Веб	В: Мобильная	Г: Enterprise
Сложность	Низкая	Средняя	Высокая	Очень высокая
Объем кода	~2000 строк	~3000 строк	~4000 строк	~6000+ строк
Необходимые знания	Java, ООП, SQL	Java, HTML/CSS, HTTP	Java + мобильный стек	Full Stack
Востребованность на рынке	Средняя	Высокая	Очень высокая	Очень высокая
Продолжительность	~80 часов	~100 часов	~120 часов	~160 часов

## ЧАСТЬ 2. ОБЯЗАТЕЛЬНОЕ ЯДРО (ДЛЯ ВСЕХ ТРАЕКТОРИЙ)

### 2.1. Этап 0: Инициация и бизнес-анализ (Недели 1-2)

#### 2.1.1. Выбор темы проекта

Критерии выбора темы:

- Реальная или приближенная к реальной предметная область
- Достаточная сложность (минимум 5-7 бизнес-сущностей)
- Наличие как типовых, так и специфических операций
- Возможность реализации за семестр

Рекомендуемые темы:

№	Категория	Проект
1	Управление ресурсами	Система управления складом
2	Управление ресурсами	Платформа аренды оборудования

3	Управление ресурсами	Система управления парком автомобилей
4	Образование	Система дистанционного обучения (LMS)
5	Образование	Платформа для проведения онлайн-курсов
6	Образование	Система записи на кружки и секции
7	Здравоохранение	Система управления медицинскими назначениями
8	Здравоохранение	Платформа записи на диагностические процедуры
9	Здравоохранение	Система мониторинга здоровья пациентов
10	Торговля и услуги	Система управления доставкой еды
11	Торговля и услуги	Платформа бронирования услуг красоты
12	Торговля и услуги	Система проката спортивного инвентаря
13	Производство	Система управления производственными заказами
14	Логистика	Платформа логистики и отслеживания грузов
15	Управление	Система управления проектами в строительстве

### 2.1.2. Результаты этапа

Артефакт	Описание	Формат
Паспорт проекта	Executive Summary (1 стр.)	Документ

Диаграмма бизнес-контекста	IDEF0 A-0	UML-диаграмма
BUC-диаграмма	Бизнес-прецеденты	UML-диаграмма
Бизнес-гlossарий	15+ терминов	Таблица
Модель бизнес-классов	Высокоуровневые сущности	UML-диаграмма
Матрица стейкхолдеров	Анализ заинтересованных лиц	Таблица
SWOT-анализ	Анализ текущего процесса	Таблица

## 2.2. Этап 1: Проектирование требований (Недели 3-4)

### 2.2.1. Переход от бизнес-модели к системным требованиям

Ключевые различия:

Аспект	Бизнес-модель (Этап 0)	Системные требования (Этап 1)
Уровень абстракции	Что должно быть сделано в бизнесе	Как система будет это делать
Акторы	Бизнес-роли	Пользователи системы
Прецеденты	Бизнес-процессы	Функциональность системы
Цель	Понимание проблемы	Создание инструкций для разработки

### 2.2.2. Результаты этапа

Артефакт	Описание	Формат
Use Case диаграмма	Системные прецеденты	UML-диаграмма
Domain Model	Концептуальная модель классов	UML-диаграмма

Спецификация прецедентов	2+ детализированных Use Case	Документ
Глоссарий (расширенный)	20+ терминов с определениями	Таблица
Таблица трассировки	Связь бизнес-требований и системных	Таблица

## 2.3. Этап 2: Архитектурное проектирование (Недели 5-6)

### 2.3.1. Архитектурные требования PCMEF

Общие принципы PCMEF:

Принцип	Описание	Проверка
Строгая иерархия слоёв	Зависимости направлены только сверху вниз: $P \rightarrow C \rightarrow M \rightarrow E \rightarrow F$	Диаграмма пакетов
Изоляция слоёв	Каждый слой отвечает только за свою зону ответственности	Код-ревью
Коммуникация через интерфейсы	Слои общаются через определённые интерфейсы	Наличие интерфейсов
Отсутствие циклических зависимостей	Граф зависимостей должен быть ациклическим	Статический анализ

Ответственность слоёв:

Слой	Ответственность	НЕ ДОЛЖЕН
------	-----------------	-----------

Presentation (P)	UI, отображение, ввод	Содержать бизнес-логику
Control (C)	Обработка запросов, валидация	Обращаться к БД напрямую
Mediator (M)	Бизнес-логика, транзакции	Знать о Presentation
Entity (E)	Состояние, бизнес-методы	Содержать логику доступа к данным
Foundation (F)	Доступ к данным, SQL	Содержать бизнес-правила

### 2.3.2. Интерфейсы между слоями

Control → Mediator (IService):

```
public interface IUserService {
 User getUserById(Long id);
 User createUser(User user);
 User authenticate(String email, String password);
 List<User> getAllUsers();
}
```

Mediator → Foundation (IRepository):

```
public interface IUserRepository {
 User findById(Long id);
 User save(User user);
 void delete(Long id);
 User findByEmail(String email);
}
```



### 2.3.3. Результаты этапа

Артефакт	Описание	Формат
Диаграмма пакетов	Структура слоёв PCMEF	UML-диаграмма
Спецификация интерфейсов	Контракты между слоями	Документ / Java-интерфейсы
Диаграмма зависимостей	Визуализация связей	UML-диаграмма
ADR	Архитектурные решения	Документ

## 2.4. Этап 3: Проектирование базы данных (Недели 7-8)

### 2.4.1. Требования к БД

- Нормализация до 3 нормальной формы
- Чёткое определение первичных и внешних ключей
- Индексы для часто запрашиваемых полей
- Ограничения целостности (NOT NULL, UNIQUE, CHECK)

### 2.4.2. Результаты этапа

Артефакт	Описание	Формат
ER-диаграмма	Логическая модель данных	UML-диаграмма
DDL-скрипты	SQL-скрипты создания таблиц	SQL
Описание стратегии ORM	Маппинг Entity → таблицы	Документ

## 2.5. Этап 4: Детальное проектирование (Недели 9-10)

### 2.5.1. Результаты этапа

Артефакт	Описание	Формат
Диаграммы последовательности	3+ сценариев	UML-диаграмма
Диаграмма классов проектирования	Детальная структура	UML-диаграмма
Спецификация методов	Сигнатуры ключевых методов	Документ

## 2.6. Этап 5: Реализация ядра (Недели 11-12)

### 2.6.1. Требования к реализации

- Полная реализация слоёв Entity и Foundation
- Реализация ключевых Use Cases в Mediator
- Модульное тестирование (JUnit, покрытие >40%)
- Соответствие архитектуре PCMEF

### 2.6.2. Результаты этапа

Артефакт	Описание	Формат
Исходный код	Реализация ядра	Java
Модульные тесты	JUnit-тесты	Java
Отчёт о покрытии	JaCoCo/ Coverage report	HTML

## 2.7. Этап 6: Рефакторинг и качество (Недели 13-14)

### 2.7.1. Обязательные паттерны рефакторинга

Паттерн	Назначение	Обязательность
---------	------------	----------------

Data Mapper	Отделение бизнес-логики от доступа к данным	Обязательно
Identity Map	Обеспечение уникальности объектов в сессии	Обязательно
Lazy Load	Отложенная загрузка связанных объектов	Рекомендуется
Unit of Work	Отслеживание изменений и пакетное сохранение	Для Enterprise

### 2.7.2. Результаты этапа

Артефакт	Описание	Формат
Отчёт статического анализа	SonarQube / Checkstyle	Документ
Рефакторинг-код	Внедрённые паттерны	Java
Обновлённые тесты	Тесты после рефакторинга	Java

## 2.8. Этап 7: Управление и документация (Недели 17-18)

### 2.8.1. Результаты этапа

Артефакт	Описание	Формат
WBS	Иерархическая структура работ	Диаграмма
Диаграмма Ганта	Календарный план	Диаграмма
Оценка COCOMO	Оценка трудозатрат	Таблица
Техническое задание	Полный документ требований	Документ

Руководство пользователя	Инструкция по работе	Документ
Руководство администратора	Инструкция по развёртыванию	Документ
Пояснительная записка	Полный отчёт о проекте	Документ
Презентация	Защита проекта	PDF/PPT

### ЧАСТЬ 3. ВАРИАТИВНАЯ ЧАСТЬ ПО ТРАЕКТОРИЯМ

#### 3.1. Траектория А: Desktopная разработка

##### 3.1.1. Требования к Desktop UI

- Минимум 5 экранов (главное окно, список, детали, создание/редактирование, отчёт)
- Реализация CRUD для основных сущностей
- Обработка событий через слушатели
- Валидация ввода на стороне клиента
- Навигация между экранами
- Использование Swing или JavaFX

##### 3.1.2. Требования к безопасности (базовые)

- Аутентификация пользователя (логин/пароль)
- Хеширование паролей (BCrypt)
- Разграничение доступа (минимум 2 роли: пользователь, администратор)

##### 3.1.3. Требования к развёртыванию (без Docker)

- Создание исполняемого JAR-файла
- Инструкция по установке и настройке

- Скрипт инициализации БД
- Список системных требований

#### 3.1.4. Бонусные задания

Бонус	Баллы	Описание
Инсталлятор	+2-4%	Создание EXE/MSI-инсталлятора
Автообновление	+2-3%	Механизм автоматического обновления
Темы оформления	+1-2%	Переключение между темами (светлая/тёмная)

### 3.2. Траектория Б: Веб-разработка

#### 3.2.1. Требования к Web UI

- Минимум 5 веб-страниц (главная, список, детали, форма, отчёт)
- Адаптивная вёрстка (mobile/desktop)
- Валидация на клиенте (JavaScript)
- AJAX-запросы для динамического обновления
- Навигация между страницами
- Использование JSP/Thymeleaf или React/Angular/Vue

#### 3.2.2. Требования к безопасности (средние)

- Аутентификация через форму логина
- Хеширование паролей (BCrypt)
- Авторизация на основе ролей (ROLE\_USER, ROLE\_ADMIN)
- Защита от XSS и SQL-инъекций
- Сессии или JWT-токены

### 3.2.3. Требования к развёртыванию (WAR)

- Сборка WAR-файла
- Инструкция по развёртыванию на Tomcat/Jetty
- Настройка подключения к БД
- Требования к серверному окружению

### 3.2.4. Бонусные задания

Бонус	Баллы	Описание
REST API	+3-5%	Дополнительное REST API с документацией
Docker	+3-5%	Контейнеризация приложения
SPA-фреймворк	+2-4%	Использование React/Vue/Angular
WebSocket	+2-3%	Real-time обновления

## 3.3. Траектория В: Мобильная разработка

### 3.3.1. Требования к мобильному интерфейсу

- Минимум 5 экранов (список, детали, создание, профиль, настройки)
- Реализация CRUD для основных сущностей
- Сетевое взаимодействие с REST API
- Обработка состояний (загрузка, ошибка, пусто)
- Локальное кэширование (оффлайн-режим)
- Навигация между экранами

Для Android Native:

- Kotlin + Jetpack Compose / XML
- ViewModel + LiveData/StateFlow
- Retrofit + OkHttp
- Room для кэширования

Для Flutter:

- Dart + Widgets
- BLoC / Riverpod / Provider
- Dio для HTTP
- SQLite / Isar / Hive

Для React Native:

- TypeScript + React Native
- Redux / MobX / Context API
- Axios / Fetch API
- AsyncStorage / SQLite

### 3.3.2. Требования к серверной части

- Java + Spring Boot
- REST API (минимум 8 эндпоинтов)
- Документация OpenAPI (Swagger UI)
- Аутентификация через JWT
- Работа с PostgreSQL
- Модульное тестирование (покрытие >40%)

Обязательные эндпоинты:

POST	/api/auth/login	Аутентификация
POST	/api/auth/register	Регистрация
GET	/api/entities	Получение списка
GET	/api/entities/{id}	Получение по ID
POST	/api/entities	Создание
PUT	/api/entities/{id}	Обновление
DELETE	/api/entities/{id}	Удаление

GET /api/entities/search Поиск с фильтрацией

### 3.3.3. Требования к безопасности

- JWT-токены для аутентификации
- Хеширование паролей (BCrypt)
- Разграничение доступа (роли: пользователь, администратор)
- Безопасное хранение токенов на клиенте
- HTTPS (эмуляция для разработки)

### 3.3.4. Требования к развёртыванию

- Сервер: сборка JAR/WAR, инструкция по запуску
- Клиент: APK (Android), IPA (iOS - опционально), или сборка для соответствующей платформы
- Документация по настройке окружения

### 3.3.5. Бонусные задания

Бонус	Баллы	Описание
Админ-панель	+2-4%	Веб-интерфейс для администратора
Push-уведомления	+2-4%	FCM / APNs
Биометрия	+1-3%	Fingerprint / Face ID для входа
Оффлайн-режим	+3-5%	Полная работа без интернета с синхронизацией
Публикация	+4-7%	Релиз в Google Play / App Store
Контейнеризация бэкенда	+2-4%	

## 3.4. Траектория Г: Enterprise (полный стек)

### 3.4.1. Требования к интерфейсам (оба типа)



- Desktop UI: 5+ экранов на JavaFX/Swing
- Web UI: 5+ страниц на React/Angular/Vue
- Единая бизнес-логика для обоих интерфейсов
- REST API для всех операций

#### 3.4.2. Требования к безопасности (полные)

- Аутентификация: JWT + refresh tokens
- Авторизация: роли и права (ROLE\_USER, ROLE\_ADMIN, ROLE\_MANAGER)
- Защита: CORS, CSRF, rate limiting
- Аудит: логирование всех действий
- HTTPS: настройка SSL/TLS

#### 3.4.3. Требования к REST API

- 15+ эндпоинтов
- Полная документация OpenAPI (Swagger UI + OpenAPI JSON)
- Пагинация и фильтрация для списков
- Версионирование API (/api/v1/...)
- Стандартные HTTP-статусы (200, 201, 400, 401, 403, 404, 500)
- Валидация входных данных

#### 3.4.4. Требования к контейнеризации (ОБЯЗАТЕЛЬНО)

- Dockerfile для серверного приложения
- Dockerfile для клиентских приложений (опционально)
- docker-compose.yml для запуска всего стека (сервер + БД + клиенты)
- Инструкция по развёртыванию с использованием Docker

#### 3.4.5. Бонусные задания

Бонус	Баллы	Описание
Мобильное приложение	+5-8%	Третий тип клиента (Android/iOS)

Kubernetes	+4-6%	Манифесты для оркестрации
CI/CD	+3-5%	GitHub Actions / GitLab CI
Мониторинг	+2-4%	Prometheus + Grafana
Кэширование	+2-3%	Redis для кэширования
GraphQL	+3-5%	Альтернатива/дополнение к REST

## ЧАСТЬ 4. ШАБЛОНЫ ДОКУМЕНТАЦИИ

### 4.1. Общие шаблоны (для всех траекторий)

#### 4.1.1. Шаблон паспорта проекта (Executive Summary)

### ПАСПОРТ ПРОЕКТА

#### 1. ОБЩАЯ ИНФОРМАЦИЯ

Название проекта: [Название]

Траектория: [Desktop / Web / Mobile / Enterprise]

Автор: [ФИО]

Дата начала: [ДД.ММ.ГГГГ]      Плановая дата завершения:  
[ДД.ММ.ГГГГ]

#### 2. БИЗНЕС-КОНТЕКСТ

Проблема: [2-3 предложения: что не так сейчас]

Решение: [2-3 предложения: что предлагает система]

#### 3. ЦЕЛИ ПРОЕКТА

- Цель 1
- Цель 2
- Цель 3

#### 4. КЛЮЧЕВЫЕ ПОКАЗАТЕЛИ УСПЕХА

- Показатель 1: [целевое значение]
- Показатель 2: [целевое значение]
- Показатель 3: [целевое значение]

#### 5. КЛЮЧЕВЫЕ РИСКИ

- Риск 1: [стратегия реагирования]
- Риск 2: [стратегия реагирования]

#### **4.1.2. Шаблон технического задания (фрагмент)**

### 2. ТРЕБОВАНИЯ К СИСТЕМЕ

#### 2.1. Требования к функциональным характеристикам

##### 2.1.1. Перечень подсистем/модулей

##### 2.1.2. Описание функций каждой подсистемы

#### 2.2. Требования к архитектуре (PCMEF)

##### 2.2.1. Описание слоёв

##### 2.2.2. Интерфейсы взаимодействия

#### 2.3. Требования к надёжности

##### 2.3.1. Время восстановления после отказа

##### 2.3.2. Сохранность информации при сбоях

#### **4.1.3. Шаблон диаграммы пакетов PCMEF**

@startuml

allowmixing

title Архитектура PCMEF

```
interface "IUserService" as IUS
interface "IUserRepository" as IUR
```

```
package "Presentation" {
 rectangle "MainWindow" as P
}
```

```
package "Control" {
 rectangle "UserController" as C
 C --> IUS : uses
}
```

```
package "Mediator" {
 rectangle "UserService" as M
 M ..|> IUS : implements
 M --> IUR : uses
}
```

```
package "Foundation" {
 rectangle "UserRepository" as F
 F ..|> IUR : implements
}
```

```
database "Database" as DB
```

```
F --> DB : JDBC
```

```
@enduml
```

## **4.2. Специфические шаблоны по траекториям**

### **4.2.1. Для мобильной траектории (Android)**

Архитектурный шаблон (PCMEF для Android):

@startuml

title Mobile PCMEF (Android)

```
package "Mobile Client" {
 package "presentation" {
 rectangle "Activity"
 rectangle "Fragment"
 rectangle "Compose"
 }
}
```

```
package "state_management" {
 rectangle "ViewModel"
 rectangle "StateFlow"
}
```

```
package "api_client" {
 rectangle "Retrofit"
 rectangle "API Interfaces"
}
```

```
package "local_cache" {
 rectangle "Room"
 rectangle "DAO"
}
}
```

```
package "Server" {
 package "control" { rectangle "Controllers" }
}
```

```

package "mediator" { rectangle "Services" }
package "entity" { rectangle "JPA Entities" }
package "foundation" { rectangle "Repositories" }
}

```

```

api_client --> control : REST
@enduml

```

### **Пример ViewModel:**

```

class EventViewModel : ViewModel() {
 private val _state = MutableStateFlow<EventState>(EventState.Loading)
 val state: StateFlow<EventState> = _state.asStateFlow()

 private val repository: EventRepository

 fun loadEvents() {
 viewModelScope.launch {
 _state.value = EventState.Loading
 try {
 val events = repository.getEvents()
 _state.value = EventState.Success(events)
 } catch (e: Exception) {
 _state.value = EventState.Error(e.message ?: "Unknown error")
 }
 }
 }
}

```

#### 4.2.2. Для мобильной траектории (Flutter)

```
class EventBloc extends Bloc<EventEvent, EventState> {
 final EventRepository repository;

 EventBloc(this.repository) : super(EventInitial()) {
 on<LoadEvents>((event, emit) async {
 emit(EventLoading());
 try {
 final events = await repository.getEvents();
 emit(EventLoaded(events));
 } catch (e) {
 emit(EventError(e.toString()));
 }
 });
 }
}
```

## **Шаблон содержания пояснительной записки курсового проекта**

### **СОДЕРЖАНИЕ**

#### **ВВЕДЕНИЕ**

Актуальность

Цель и задачи

Объект и предмет исследования

#### **1. АНАЛИТИЧЕСКАЯ ЧАСТЬ**

1.1. Описание предметной области

1.2. Анализ бизнес-процессов (IDEF0)

1.3. SWOT-анализ текущего состояния

1.4. Анализ аналогов и конкурентов

1.5. Обоснование необходимости разработки

## 1.6. Экономическое обоснование (ROI)

## 2. ПРОЕКТНАЯ ЧАСТЬ

### 2.1. Модель требований к ПО

#### 2.1.1. Use Case диаграмма

#### 2.1.2. Спецификация прецедентов

#### 2.1.3. Глоссарий терминов

### 2.2. Модель предметной области

#### 2.2.1. Domain Model (диаграмма классов)

#### 2.2.2. Описание сущностей и атрибутов

#### 2.2.3. Бизнес-правила

### 2.3. Архитектурное проектирование

#### 2.3.1. Выбор архитектурного стиля

#### 2.3.2. Диаграмма пакетов (PCMEF)

#### 2.3.3. Описание слоёв и их ответственности

#### 2.3.4. Архитектурные решения (ADR)

### 2.4. Проектирование базы данных

#### 2.4.1. ER-диаграмма

#### 2.4.2. Физическая модель данных

#### 2.4.3. DDL-скрипты

### 2.5. Детальное проектирование

#### 2.5.1. Диаграммы последовательности

#### 2.5.2. Диаграммы классов проектирования

#### 2.5.3. Применение паттернов GoF

## 3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ

### 3.1. Реализация бизнес-логики

#### 3.1.1. Структура проекта

#### 3.1.2. Классы-сущности

#### 3.1.3. Слой доступа к данным



#### 3.1.4. Слой управления

### 3.2. Рефакторинг и оптимизация

#### 3.2.1. Статический анализ кода

#### 3.2.2. Применение паттернов (Data Mapper, Identity Map)

#### 3.2.3. Оптимизация запросов

### 3.3. Пользовательский интерфейс

#### 3.3.1. Desktop приложение

#### 3.3.2. Web приложение

#### 3.3.3. Сравнение реализаций

### 3.4. Безопасность и транзакции

#### 3.4.1. Аутентификация и авторизация

#### 3.4.2. Управление транзакциями

#### 3.4.3. Защита от атак

### 3.5. REST API

#### 3.5.1. Спецификация OpenAPI

#### 3.5.2. Реализация контроллеров

#### 3.5.3. Тестирование API

## 4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА

### 4.1. Модульное тестирование

### 4.2. Интеграционное тестирование

### 4.3. Системное тестирование

### 4.4. Нагрузочное тестирование

### 4.5. Результаты тестирования

## 5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ

### 5.1. Контейнеризация (Docker)

### 5.2. Инструкция по установке

### 5.3. Инструкция по настройке

### 5.4. Требования к окружению

## 6. УПРАВЛЕНИЕ ПРОЕКТОМ

6.1. WBS (иерархическая структура работ)

6.2. Диаграмма Ганта

6.3. Оценка трудозатрат (COCOMO)

6.4. Управление рисками

## ЗАКЛЮЧЕНИЕ

Выводы

Результаты

Перспективы развития

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

## ПРИЛОЖЕНИЯ

Приложение А. Листинги кода

Приложение Б. Скриншоты интерфейсов

Приложение В. Результаты тестирования

## ЧАСТЬ 5. КРИТЕРИИ ОЦЕНКИ

### 5.1. Общие критерии (70% — для всех траекторий)

Категория	Критерий	Вес	Описание
Бизнес-анализ (10%)	Полнота бизнес-модели	3%	IDEF0, BUC, глоссарий, бизнес-классы
	Качество анализа	4%	SWOT, стейкхолдеры, CJM проработаны
	Экономическое обоснование	3%	ROI рассчитан реалистично

Требования (10%)	Use Case диаграмма	3%	Полнота, корректность отношений
	Domain Model	4%	Сущности, атрибуты, связи, бизнес-правила
	Спецификация прецедентов	3%	Детализация 2+ use cases
Архитектура (15%)	Обоснование выбора PCMEF	3%	Чёткое объяснение адаптации
	Диаграмма пакетов	5%	Все слои, корректные зависимости
	Интерфейсы между слоями	4%	IService, IRepository определены
	Отсутствие циклов	3%	Проверка зависимостей
Реализация ядра (15%)	Структура проекта	3%	Соответствие PCMEF
	Классы- сущности	4%	С бизнес-методами, не анемичные
	Слой доступа к данным	4%	Реализация репозитория
	Модульное тестирование	4%	Покрытие >40%
Рефакторинг (10%)	Статический анализ	3%	Исправление критических замечаний
	Data Mapper	4%	Отделение доступа к данным
	Identity Map	3%	Уникальность объектов
Документация (10%)	Техническое задание	2%	Полнота

	Руководство пользователя	2%	Понятность
	Пояснительная записка	3%	Структура, полнота
	Планирование (WBS, Ганта)	3%	Качество

## 5.2. Специфические критерии по траекториям (30%)

### 5.2.1. Для траектории А (Десктоп)

Критерий	Вес	Описание
Desktop UI	15%	5+ экранов, навигация, обработка событий
Аутентификация	5%	Базовая безопасность
Развёртывание	5%	JAR + инструкция
UX/дизайн	5%	Удобство использования

### 5.2.2. Для траектории Б (Веб)

Критерий	Вес	Описание
Web UI	12%	5+ страниц, адаптивность, валидация
REST API	8%	8+ эндпоинтов
Безопасность	5%	Аутентификация, авторизация, защита
Развёртывание	5%	WAR + инструкция

### 5.2.3. Для траектории В (Мобильная)

Критерий	Вес	Описание
Мобильный UI	10%	5+ экранов, навигация, Material Design

Серверная часть	8%	Java + Spring Boot, качество кода
REST API	5%	8+ эндпоинтов, OpenAPI
JWT-безопасность	3%	Аутентификация, роли
Оффлайн-режим	4%	Кэширование, работа без интернета

#### 5.2.4. Для траектории Г (Enterprise)

Критерий	Вес	Описание
Desktop + Web UI	8%	Оба типа интерфейсов
REST API	6%	15+ эндпоинтов, полная документация
Безопасность	5%	JWT, роли, аудит, защита
Контейнеризация	6%	Docker + docker-compose
Тестирование	5%	Интеграционные тесты

#### 5.3. Бонусные баллы

Бонус	Макс. балл	Описание
Инновационность решения	+5%	Применение современных технологий
Промышленный уровень кода	+5%	CI/CD, мониторинг, продвинутые практики
Паттерны высокого уровня	+3%	CQRS, Event Sourcing, DDD
Публичный репозиторий	+2%	GitHub с документацией
Дополнительный клиент	+7%	Третий тип интерфейса

#### 5.4. Штрафы

Нарушение	Штраф
-----------	-------

Нарушение сроков сдачи	-5% за каждую неделю
Отсутствие Git	-10%
Несоответствие архитектуре PCMEF	-20%
Отсутствие сервера (для Mobile/Web/Enterprise)	-30% (недопуск)
Плагат	0 баллов

#### Список рекомендуемой литературы

1. Мацяшек, Л. Практическая программная инженерия на основе учебного примера / Л. Мацяшек. — М.: Вильямс, 2021.
2. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. — 10-е изд. — М.: Вильямс, 2018.
3. Мартин, Р. Чистая архитектура / Р. Мартин. — СПб.: Питер, 2018.
4. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. — М.: Вильямс, 2016.